

```
In [2]: # this line makes figures interactive in Jupyter notebooks
%matplotlib inline
from matplotlib import pyplot as plt

import numpy as np
from scipy.optimize import root_scalar
from scipy import constants
import cantera as ct

from pint import UnitRegistry
ureg = UnitRegistry()
Q_ = ureg.Quantity

# for convenience:
def to_si(quant):
    '''Converts a Pint Quantity to magnitude at base SI units.
    ...

    return quant.to_base_units().magnitude
```

```
In [ ]:
```

```
In [3]: def vertical_launch(t, y, spec_impulse, mass_prop, time_burn, gamma):
    '''Evaluates system of time derivatives for velocity, altitude, and mass.
    ...

    radius_earth = 6378.388 * 1e3
    gravity_ref = 9.80665
    density_ref = 1.225
    pressure_ref = 101325
    height_den = 10400
    height_pres = 8400

    v = y[0]
    h = y[1]
    m = y[2]

    gravity = gravity_ref * (radius_earth / (radius_earth + h))**2
    density = density_ref * np.exp(-h / height_den)
    pressure = pressure_ref * np.exp(-h / height_pres)

    mach = v / np.sqrt(gamma * pressure / density)
    #area = np.pi * diameter**2 / 4
    #drag = 0.5 * density * v**2 * area * coef_drag(mach)

    dmdt = -mass_prop / time_burn
    dvdt = (-spec_impulse * gravity_ref / m) * dmdt - gravity
    dhdt = v

    return [dvdt, dhdt, dmdt]
```

```
In [4]: def get_area_ratio(pressure_ratio, gamma):
    '''Calculates area ratio based on specific heat ratio and pressure ratio.
    pressure ratio: chamber / exit
    area ratio: exit / throat
```

```

'''
return (
    np.power(2 / (gamma + 1), 1/(gamma-1)) *
    np.power(pressure_ratio, 1 / gamma) *
    np.sqrt((gamma - 1) / (gamma + 1) /
            (1 - np.power(pressure_ratio, (1 - gamma)/gamma))
    )

def root_area_ratio(pressure_ratio, area_ratio, gamma):
    ''' Returns zero for a given area ratio, pressure ratio, and gamma.
    pressure ratio: chamber / exit
    area ratio: exit / throat
    '''
    return area_ratio - get_area_ratio(pressure_ratio, gamma)

def get_thrust_coeff(pressure_ratio, gamma):
    '''Calculates thrust coefficient for optimum expansion.
    pressure ratio: chamber / exit
    area ratio: exit / throat
    '''
    return np.sqrt(
        2 * np.power(gamma, 2) / (gamma - 1) *
        np.power(2 / (gamma + 1), (gamma + 1)/(gamma - 1)) *
        (1 - np.power(1.0 / pressure_ratio, (gamma - 1)/gamma))
    )

def get_cstar(Tc, MW, gamma):
    '''Calculates cstar using chamber properties.'''
    return (
        np.sqrt(Ru * Tc / (gamma * MW)) *
        (2 / (gamma + 1))**(-0.5*(gamma + 1)/(gamma - 1))
    )

```

```

In [5]: delta_velocity = Q_(2300, 'm/s')
        mpl = Q_(100000, 'kg')
        propellant_mass_fraction = 0.9
        burn_time = Q_(80, 's')
        Pc_initial = Q_(1000, 'psi')
        a = Q_(0.04, 'in/s / psi**0.3')
        n = 0.3
        g0 = Q_(9.81, 'm/s^2')

```

```

In [8]: mixture_ratios = np.arange(2, 3.05, 0.1) #Mixture ratios tested
        with open('HTPBALAP_1.txt', 'r') as f:
            lines = f.readlines()

        c_star_test = []

        for line in lines:
            # ignore blank lines
            if not line.strip():
                continue

            words = line.split()

```

```

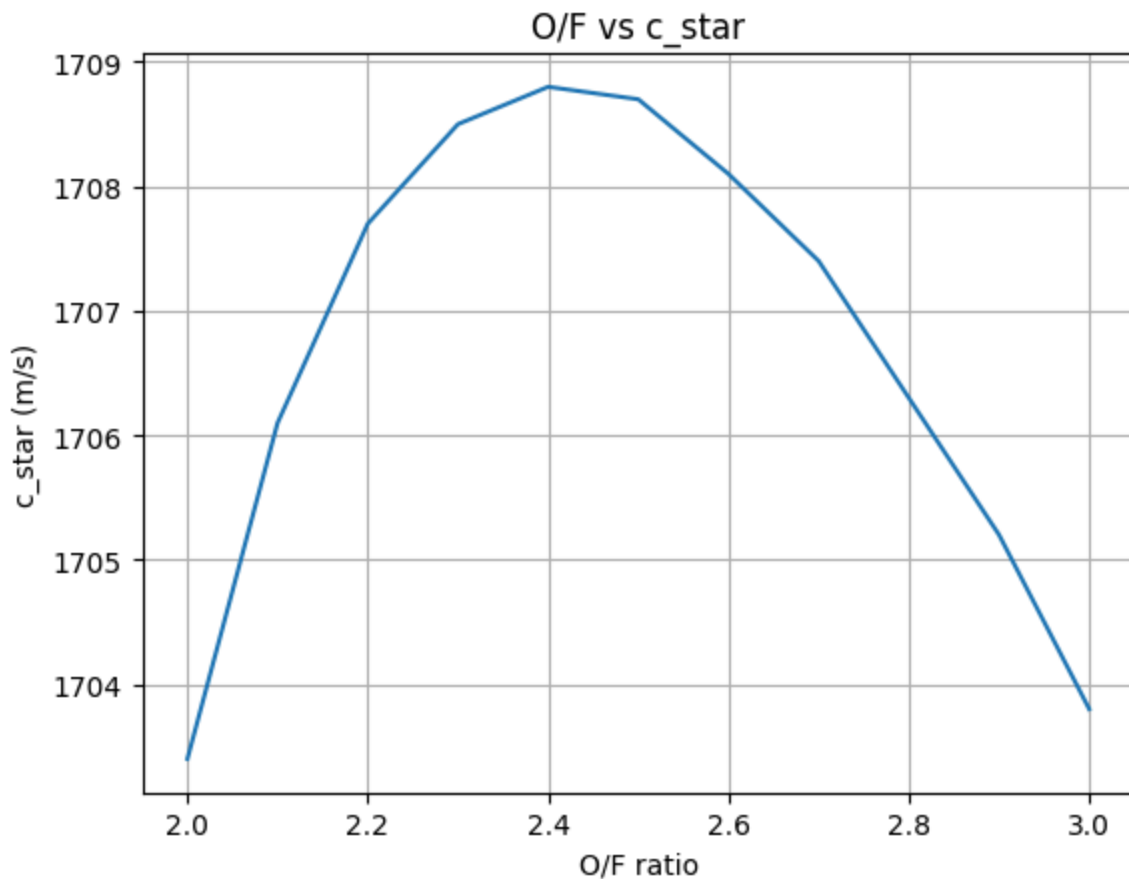
if words[0] == 'CSTAR,':
    c_star_test.append(float(words[3]))

c_star_test = Q_(np.array(c_star_test), 'm/s')

plt.plot(mixture_ratios, c_star_test.magnitude)
plt.grid(True)
plt.xlabel('O/F ratio')
plt.ylabel('c_star (m/s)')
plt.title('O/F vs c_star')
plt.show

```

Out[8]: <function matplotlib.pyplot.show(close=None, block=None)>



```

In [9]: #Propellant 1: AL + HTPB & Ammonium Perchlorate
#No Analysis of O/F ratio, as it is not applicable.
#For CEA, use butadiene (C4H6) as it has similar properties and HTPB isn't available
#The O/F of 2.125 is assuming the fuel binder and fuel are combined into a fuel mass
r = 2.4 # ~ 70% AP, ~19% AL, 11% HTPB

Pe = Q_(14.7, 'psi') #From https://www.noaa.gov/jetstream/atmosphere/air-pressure
pressure_ratio = Pc_initial / Pe

gamma = 1.1318
c_star = Q_(1708.8, 'm/s')
Isp = Q_((2812.9), 'm/s') / g0

```

```
area_ratio = get_area_ratio(pressure_ratio, gamma)
CF = get_thrust_coeff(pressure_ratio, gamma)
```

```
In [13]: Al_density = Q_(2700, 'kg/m^3') # https://pubchem.ncbi.nlm.nih.gov/compound/5359268
# print(Al_density)
AP_density = Q_(1950, 'kg/m^3') # https://pubchem.ncbi.nlm.nih.gov/compound/Ammoniu
# print(AP_density.to('kg/m^3'))
HTPB_density = Q_(0.902, 'g/cm^3') # https://www.jstage.jst.go.jp/article/rheology/
# print(HTPB_density.to('kg/m^3'))

# Solving for propellant density
propellant_density = 0.70 * AP_density + 0.19 * Al_density + 0.11 * HTPB_density
print('The propellant density is calculated to be', propellant_density.to('kg/m^3'))
```

The propellant density is calculated to be 1977.22 kilogram / meter ** 3

```
In [14]: print('The Isp is', Isp)
print('The area ratio is', area_ratio)
# print('The regression rate is', rdot)
print('The thrust coefficient is', CF)
```

The Isp is 286.7380224260958 second

The area ratio is 10.233075920682616 dimensionless

The thrust coefficient is 1.6395591701917767 dimensionless

```
In [15]: # For the masses of propellants, we are going to use Tsiokvsky Rocket equation
mo_mf = np.exp(delta_velocity / (Isp * g0))
# Using the propellant mass fraction of 0.9, we can use systems of equations to solve
ms = mpl * (mo_mf - 1) / (10 - mo_mf)
mp = 9 * ms

fuel_mass = mp / (1 + r)
oxidizer_mass = fuel_mass * r

# fuel_volume = fuel_mass / fuel_density
# oxidizer_volume = oxidizer_mass / oxidizer_density
# total_volume = fuel_volume + oxidizer_volume
total_volume = mp / propellant_density

# We are given the time burn, so we can find the mass flow rate
mdot_propellant_avg = mp / burn_time

# We also can find the liftoff mass
liftoff_mass = mpl + ms + mp
# Requirements ask for twice the booster liftoff mass for average thrust
thrust_requested = liftoff_mass * g0 * 2 # Is it liftoff mass or is it the mass of the
# We can also use the values we have to calculate the area of throat required for the

# If we assume a constant mass flow rate, our thrust at any time is
Thrust_produced = Isp * g0 * mdot_propellant_avg
```

```
In [16]: print('The starting mass to final mass ratio is', f'{mo_mf.magnitude:.3}')
print('The structural mass is', f'{ms:.6}')
print('The propellant mass is', f'{mp:.10}')
print('The total volume needed is', f'{total_volume}')
print('The average mass flow rate of propellant is', f'{mdot_propellant_avg:.6}')
```

```

print('The total liftoff mass is', liftoff_mass)
print('The requested thrust is', f'{thrust_requested.to('N'):.8}')
print('The Thrust produced is', Thrust_produced.to('N')) #The resulting thrust is L

```

The starting mass to final mass ratio is 2.27
 The structural mass is 16357.2 kilogram
 The propellant mass is 147214.7027 kilogram
 The total volume needed is 74.45539834551039 meter ** 3
 The average mass flow rate of propellant is 1840.18 kilogram / second
 The total liftoff mass is 263571.89190745563 kilogram
 The requested thrust is 5171280.5 newton
 The Thrust produced is 5176252.9658979215 newton

In [17]: *#Nozzle Design*

```

At = Thrust_produced / (CF*Pc_initial)
Rt = np.sqrt(At / np.pi)
Dt = 2 * Rt

Ae = At * area_ratio
Re = np.sqrt(Ae / np.pi)
De = 2 * Re

nozzle_length_ratio = 4

Ln = Dt * nozzle_length_ratio

```

In [18]:

```

print('Throat Area', At.to('m^2'))
print('Throat Radius', Rt.to('m'))
print('Throat Diameter', Dt.to('m'))
print('Exit Area', Ae.to('m^2'))
print('Exit Radius', Re.to('m'))
print('Exit Diameter', De.to('m'))
print('Nozzle Length', Ln.to('m'))

```

Throat Area 0.4578987045678297 meter ** 2
 Throat Radius 0.38177700891317573 meter
 Throat Diameter 0.7635540178263515 meter
 Exit Area 4.685712207824821 meter ** 2
 Exit Radius 1.221273318943273 meter
 Exit Diameter 2.442546637886546 meter
 Nozzle Length 3.054216071305406 meter

In [19]: *#Attempting to set up chamber to hold propellant*

```

Df = Q_(3, 'm')
rdot = a * Pc_initial**n
web_thickness = burn_time * rdot

Rf = Df / 2

Di = Df - 2 * web_thickness
Ri = Di / 2

L = total_volume / (np.pi * (Rf**2 - Ri**2))
#Using the propellant volume, we can calculate the Length of the chamber required

```

```
In [20]: print('rdot', rdot)
print('web thickness', web_thickness.to('m'))
print('outer diameter', Df)
print('outer radius', Rf)
print('inner diameter', Di)
print('inner radius', Ri)
print('chamber length', L.to('m'))
```

```
rdot 0.31773129388971255 inch / second
web thickness 0.6456299891838959 meter
outer diameter 3 meter
outer radius 1.5 meter
inner diameter 1.7087400216322082 meter
inner radius 0.8543700108161041 meter
chamber length 15.59150027242837 meter
```

```
In [21]: from scipy.integrate import solve_ivp
#print(Isp, mp, burn_time, gamma)
sol = solve_ivp(
    vertical_launch, [0, burn_time.magnitude], [0, 0, liftoff_mass.to('kg').magnitude],
    args=(Isp.magnitude, mp.to('kg').magnitude, burn_time.magnitude, gamma),
    dense_output=True
)
```

```
In [22]: time = np.linspace(0, burn_time.magnitude, 100)
z = sol.sol(time)

acceleration = np.gradient(z[0, :], time)

fig, axes = plt.subplots(3, 1)

axes[1].plot(time, z[0, :])
axes[1].set_ylabel('Velocity (m/s)')
axes[1].grid(True)

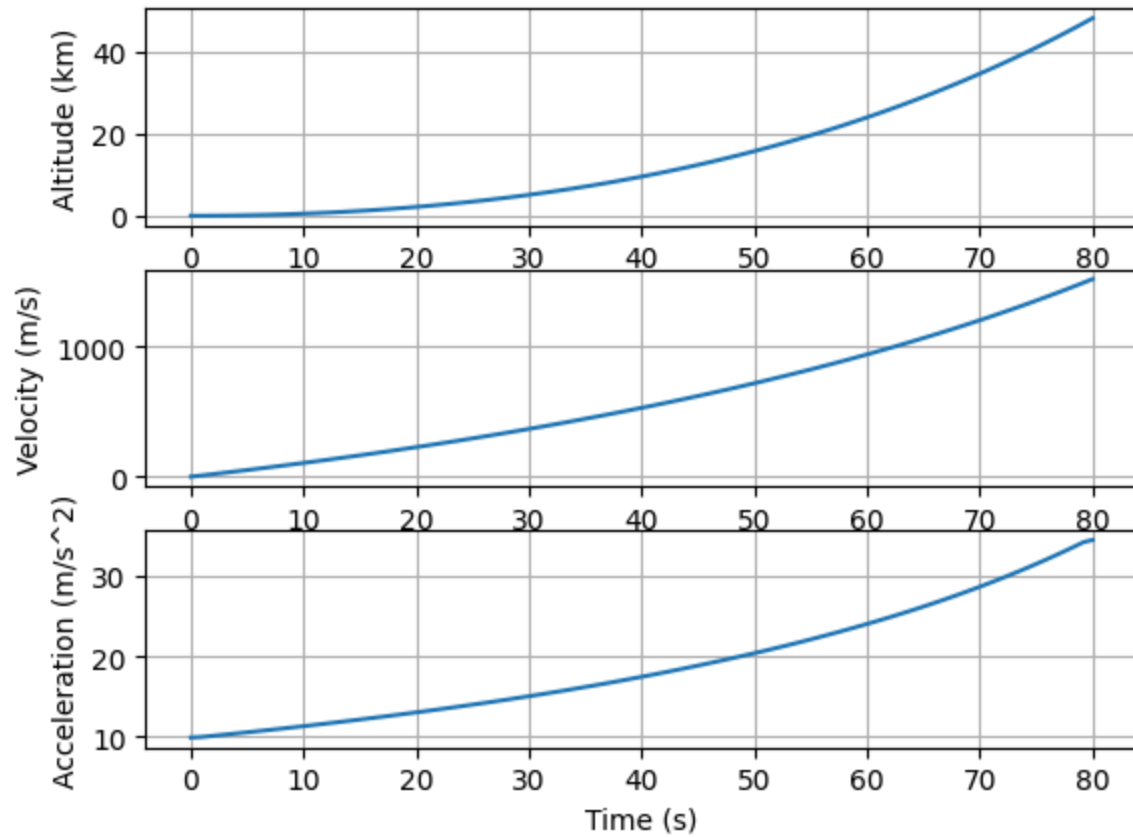
axes[0].plot(time, z[1, :] / 1000)
axes[0].set_ylabel('Altitude (km)')
axes[0].grid(True)

axes[2].plot(time, acceleration)
axes[2].set_xlabel('Time (s)')
axes[2].set_ylabel('Acceleration (m/s^2)')
axes[2].grid(True)

plt.suptitle('Rocket Performance for HTPB + AL + AP', fontsize=16, fontweight='bold')

plt.show()
#Remove drag component.
```

Rocket Performance for HTPB + AL + AP



In []: